

Name: Sanskruti Dahiphale

PRN: 123B1B007

Assignment No. 6

Title:

Implement a backtracking algorithm to solve a Sudoku puzzle. The program should read an incomplete puzzle grid and output a valid solution, if one exists. The program should also handle edge cases such as unsolvable puzzles and be tested with multiple inputs.

Objectives:

- To apply the Backtracking strategy to solve constraint satisfaction problems.
- To systematically search for solutions and eliminate invalid paths early.

Course Outcome Mapping:

CO4: Apply Backtracking approach to implement solutions for combinatorial problems.

Theory:

Backtracking Algorithm Strategy:

Backtracking is a systematic technique used to solve **combinatorial and constraint satisfaction problems** by exploring all possible solutions in a structured manner. It incrementally builds a solution and immediately discards any partial solution that violates the given constraints.

In the case of **Sudoku**, the algorithm works by filling empty cells one at a time with values from 1 to 9. For each assignment, it verifies whether the number satisfies all the rules of Sudoku (no repetition in the same row, column, or 3×3 grid).

- If the placement is valid, the algorithm proceeds to the next empty cell.
- If it violates any constraint, the number is rejected and the next possible value is tried.
- If no valid number can be placed in a cell, the algorithm backtracks to the previous cell and changes its value.

This process continues recursively, forming a **state space tree**, where each node represents a partial configuration of the board. By exploring only feasible paths and abandoning invalid ones early, backtracking efficiently arrives at the correct solution.

Sudoku Constraints:

- Each number from 1 to 9 must appear only once in each row.
- Each number from 1 to 9 must appear only once in each column.
- Each number from 1 to 9 must appear only once in each 3×3 subgrid.

Example:**Input Sudoku Grid:**

5	3	0	0	7	0	0	0	0
6	0	0	1	9	5	0	0	0
0	9	8	0	0	0	0	6	0
8	0	0	0	6	0	0	0	3
4	0	0	8	0	3	0	0	1
7	0	0	0	2	0	0	0	6
0	6	0	0	0	0	2	8	0
0	0	0	4	1	9	0	0	5
0	0	0	0	8	0	0	7	9

Output (Solved Sudoku):

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5

3	4	5	2	8	6	1	7	9
---	---	---	---	---	---	---	---	---

Algorithm:

Backtracking Procedure for Solving Sudoku:

1. Identify the next empty cell in the grid.
2. Attempt to fill the cell with digits from 1 to 9 sequentially.
3. For each digit, verify whether it satisfies all Sudoku constraints:
 - The digit should not already exist in the same row.
 - The digit should not be present in the same column.
 - The digit should not appear in the corresponding 3×3 subgrid.
4. If a valid digit is found, assign it to the cell and proceed to solve the next empty cell.
5. If the digit is not valid, continue trying the next possible value.
6. If none of the digits (1–9) fit, undo the previous assignment (backtrack) and try a different value in the previous cell.
7. Continue this process recursively until all cells are correctly filled or it is determined that no solution exists.

Time Complexity:

- Worst Case: $O(9^n)$

Explanation:

- Each empty cell can take up to 9 values.
- The algorithm explores all possible combinations in the worst case.
- However, backtracking significantly reduces unnecessary computations by pruning invalid paths early.

Code:

```
# Function to print Sudoku
def print_board(board):
    for row in board:
        print(row)

# Check if placing number is valid
def is_safe(board, row, col, num):
    # Check row
    for x in range(9):
        if board[row][x] == num:
            return False

    # Check column
    for x in range(9):
```

```

if board[x][col] == num:
    return False

# Check 3x3 grid
start_row = row - row % 3
start_col = col - col % 3

for i in range(3):
    for j in range(3):
        if board[start_row + i][start_col + j] == num:
            return False

return True

# Solve Sudoku using Backtracking
def solve_sudoku(board):
    for row in range(9):
        for col in range(9):
            if board[row][col] == 0:

                for num in range(1, 10):
                    if is_safe(board, row, col, num):
                        board[row][col] = num

                if solve_sudoku(board):
                    return True

            board[row][col] = 0 # Backtrack
            return False
        return True

# ----- PREDEFINED INPUT -----

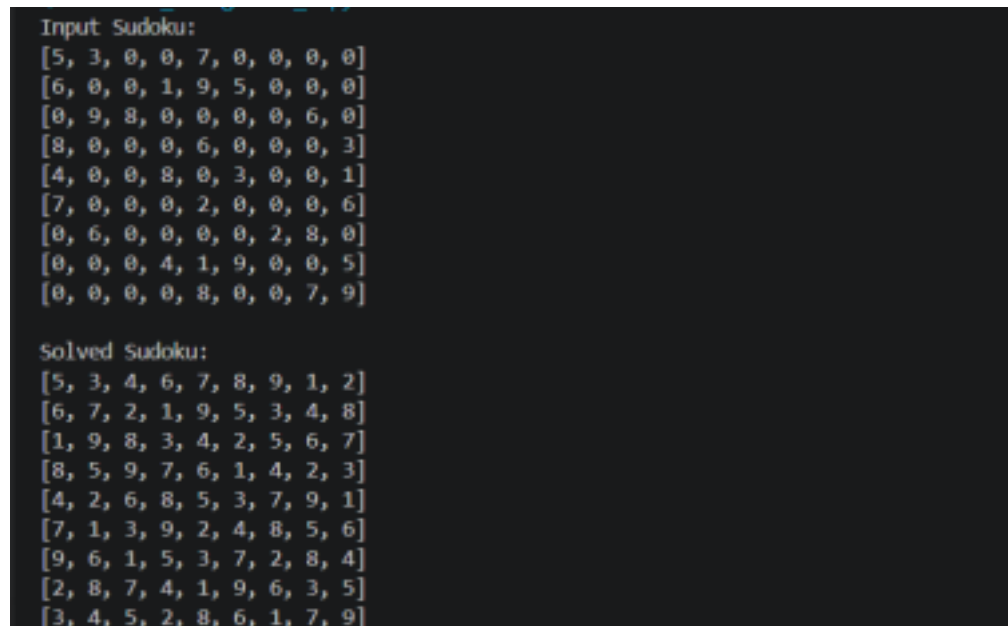
board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

print("Input Sudoku:")
print_board(board)

```

```
if solve_sudoku(board):
    print("\nSolved Sudoku:")
    print_board(board)
else:
    print("\nNo solution exists!")
```

Output:



```
Input Sudoku:
[5, 3, 0, 0, 7, 0, 0, 0, 0]
[6, 0, 0, 1, 9, 5, 0, 0, 0]
[0, 9, 8, 0, 0, 0, 0, 6, 0]
[8, 0, 0, 0, 6, 0, 0, 0, 3]
[4, 0, 0, 8, 0, 3, 0, 0, 1]
[7, 0, 0, 0, 2, 0, 0, 0, 6]
[0, 6, 0, 0, 0, 0, 2, 8, 0]
[0, 0, 0, 4, 1, 9, 0, 0, 5]
[0, 0, 0, 0, 8, 0, 0, 7, 9]

Solved Sudoku:
[5, 3, 4, 6, 7, 8, 9, 1, 2]
[6, 7, 2, 1, 9, 5, 3, 4, 8]
[1, 9, 8, 3, 4, 2, 5, 6, 7]
[8, 5, 9, 7, 6, 1, 4, 2, 3]
[4, 2, 6, 8, 5, 3, 7, 9, 1]
[7, 1, 3, 9, 2, 4, 8, 5, 6]
[9, 6, 1, 5, 3, 7, 2, 8, 4]
[2, 8, 7, 4, 1, 9, 6, 3, 5]
[3, 4, 5, 2, 8, 6, 1, 7, 9]
```

Conclusion:

The Backtracking algorithm is an effective method for solving Sudoku puzzles by exploring all possible solutions and eliminating invalid ones. It ensures that a correct solution is found if it exists.

Although the worst-case time complexity is exponential, backtracking reduces the search space significantly by pruning invalid paths. Therefore, it is highly suitable for solving combinatorial and constraint satisfaction problems like Sudoku.